Experiment - 2

AIM- To interface a push button as digital input with an LED as digital output for simple sensor-actuator control.

APPARATUS- STM32 Discovery Kit, STM32 CUBE IDE, STM32 CubeMX.

PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L475VGT6.
3. In system Core select GPIO and configure PC13 as a GPIO input pin and PB14 as GPIO output pin as shown below:

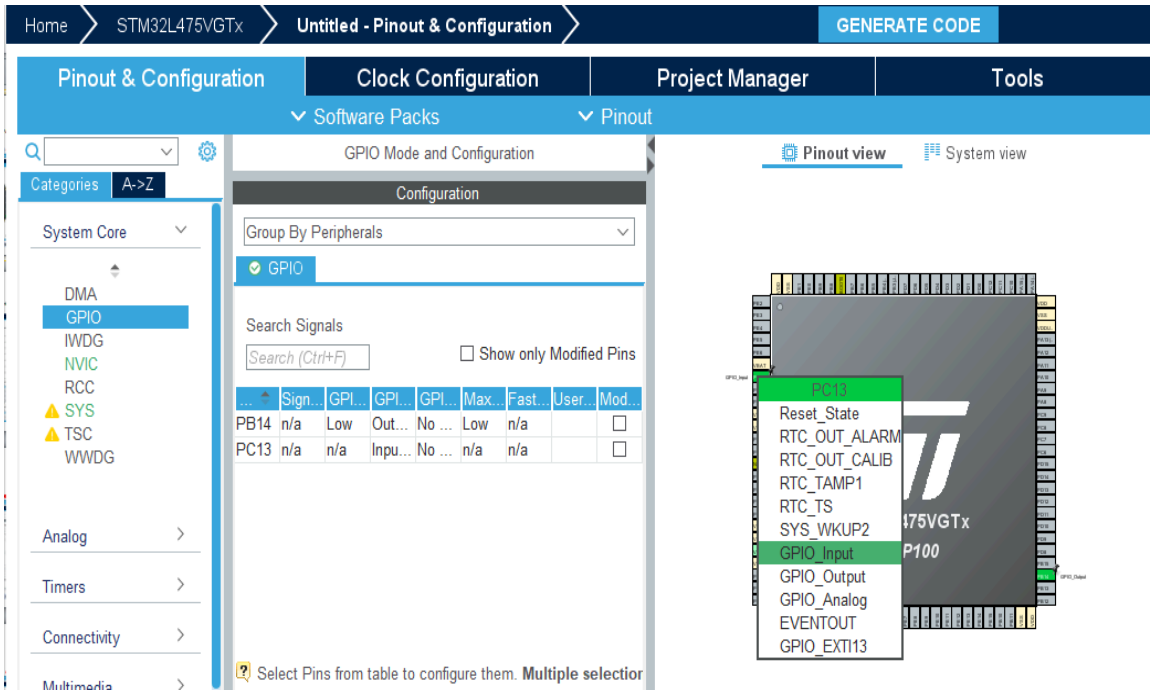

Fig-1.1 – STM32 Board Pinout and Configuration

4. **Clock Selection**: Go to RCC and configure clock as “By pass Clock Source” as shown below:

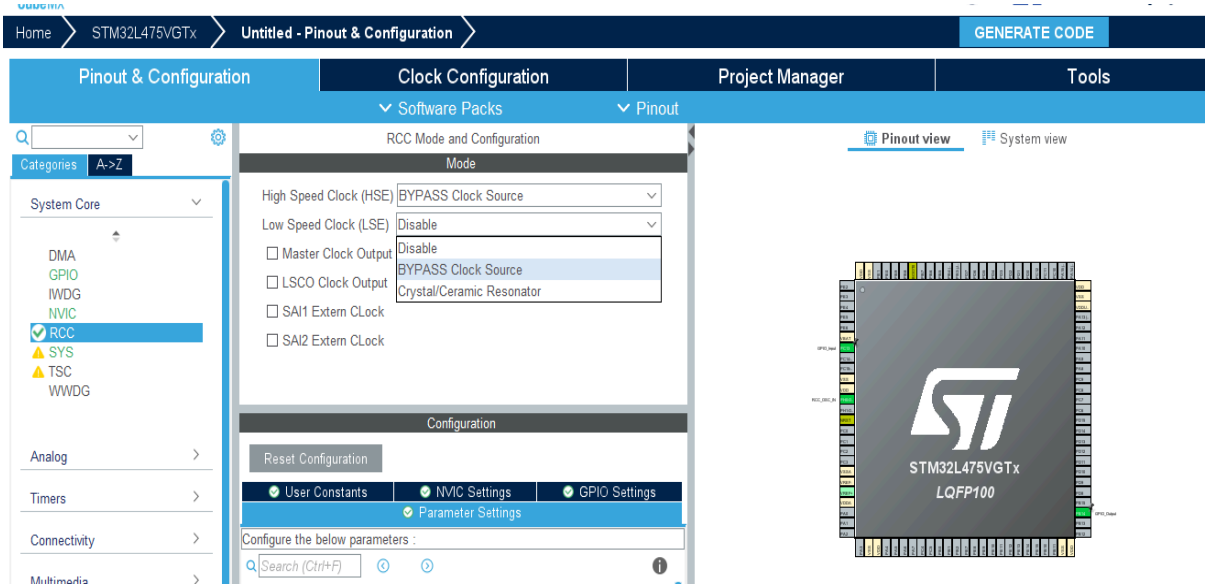

Fig-1.2 – Clock Selection

5. **Clock Configuration:** Configure the clock as shown below to get a maximum frequency of 80MHz.

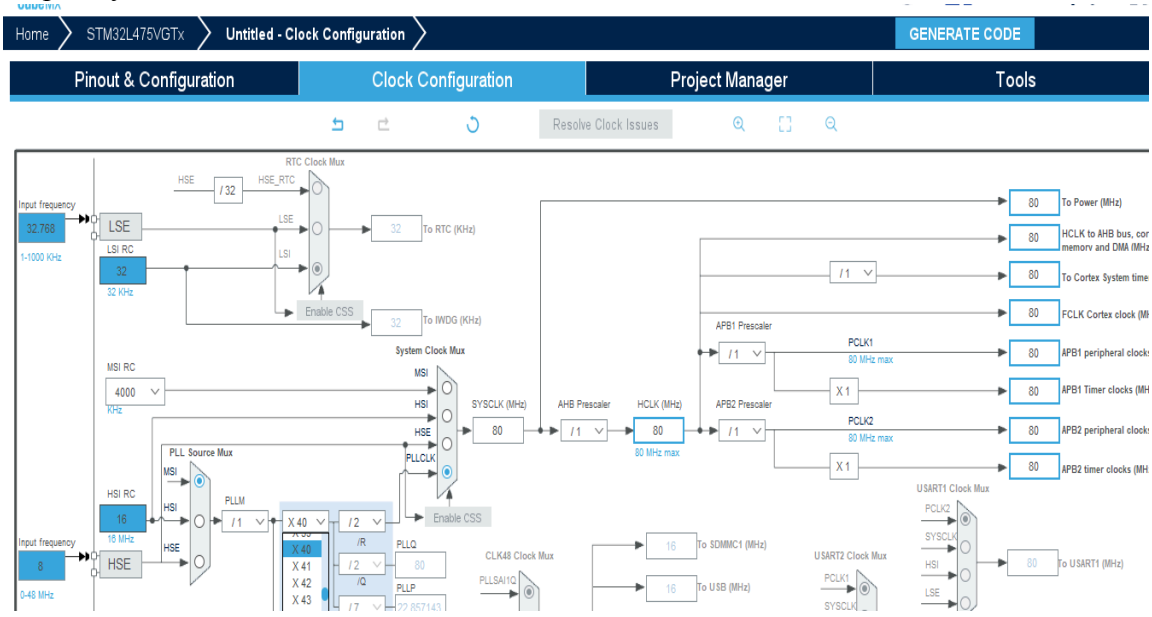

Fig-1.3 – Clock Configuration

6. Write Project name and select IDE as shown below:

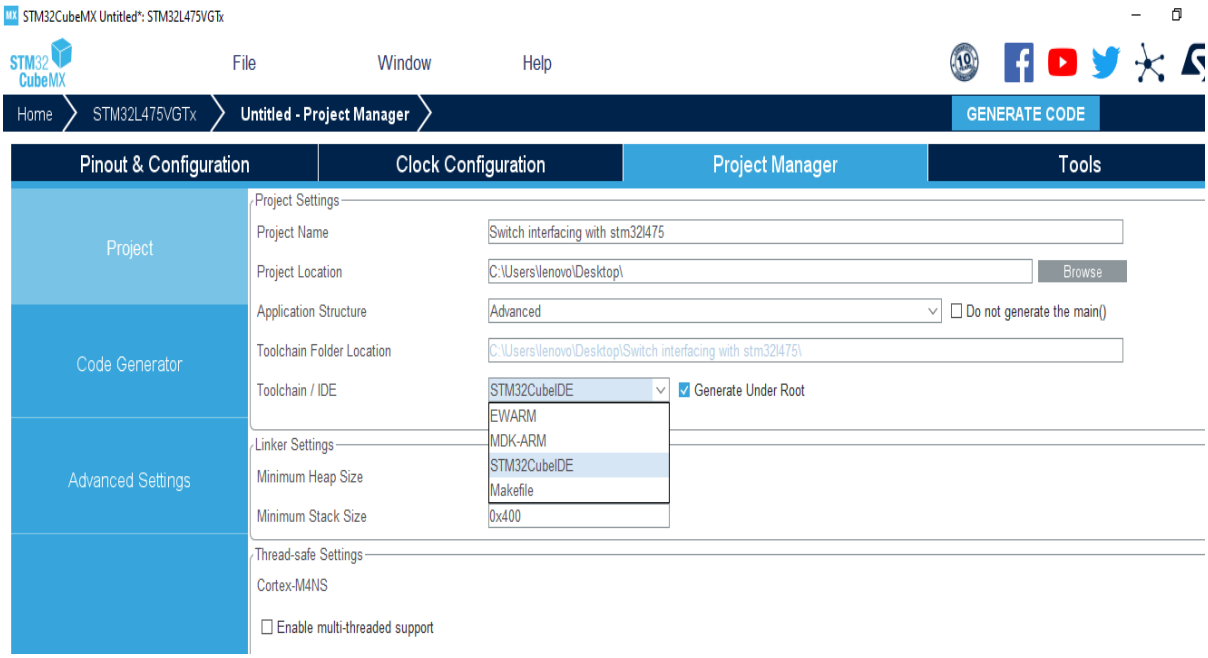

Fig-1.4 – Project Name and IDE

7. Click on generate code.

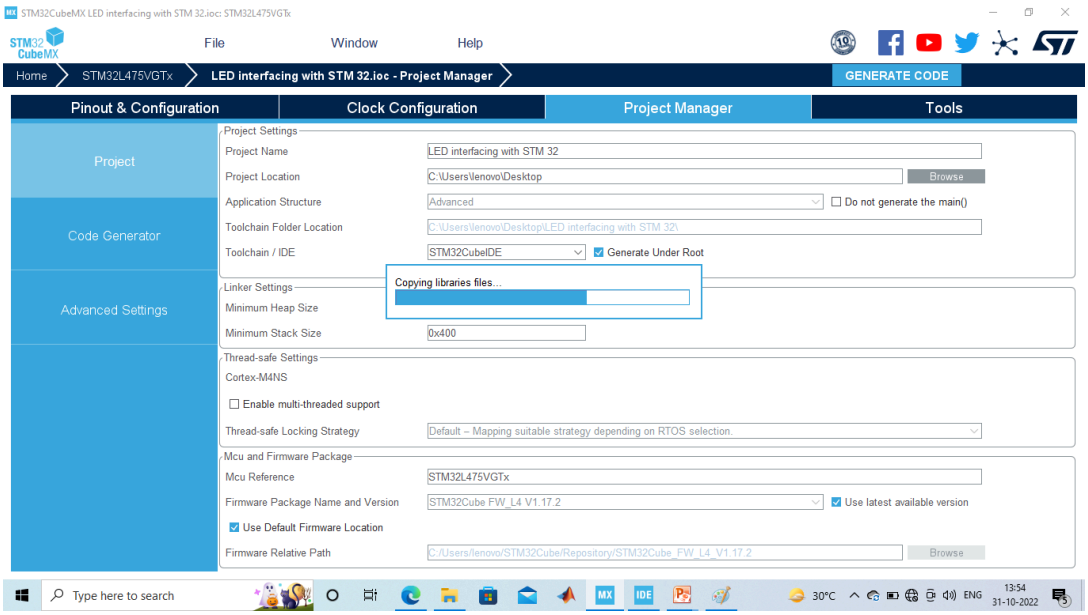

Fig-1.5 – Generate Code

8. Click on open project.

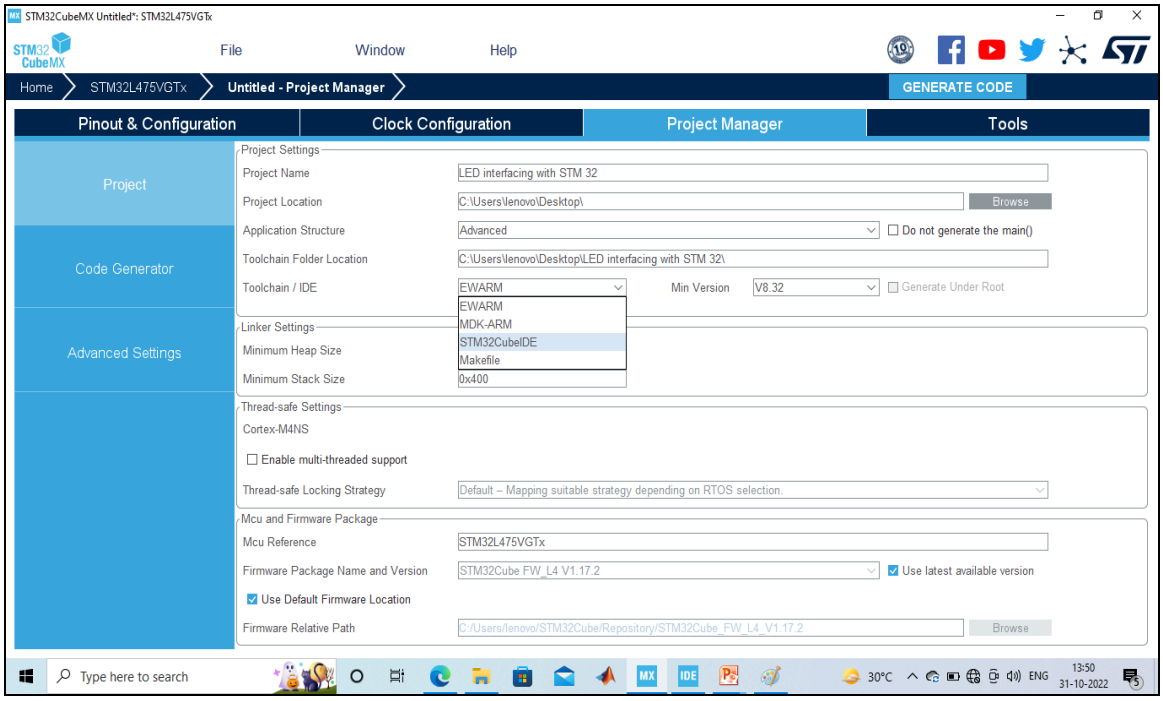

Fig:1.6 – Project Set up

9. Click 'OK' on the generated popup showing "successfully imported the project on the work space" and the project will be successfully added to the cube IDE.
10. Open the main source code on Cube IDE by clicking on main.c
11. After configuration write only below two instructions in the user while loop and compile the program and ensure there will be zero error after compiling.

```
if(HAL_GPIO_ReadPin (GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET)
    {
     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_SET);
    }
                HAL_Delay(100);
                 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_RESET);
```

12. Click build and debug as shown below:

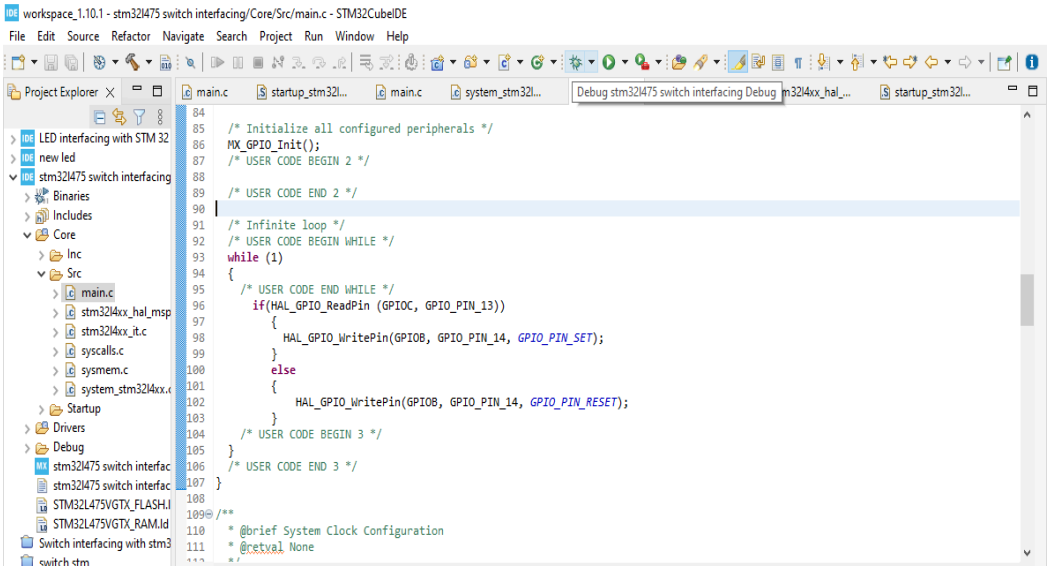

Fig: 1.7 - Program for executing the experiment

13. To blink the connected LED on the board, connect the board, click on build, click ‘ok’ and click ‘switch’ on popups as shown below:

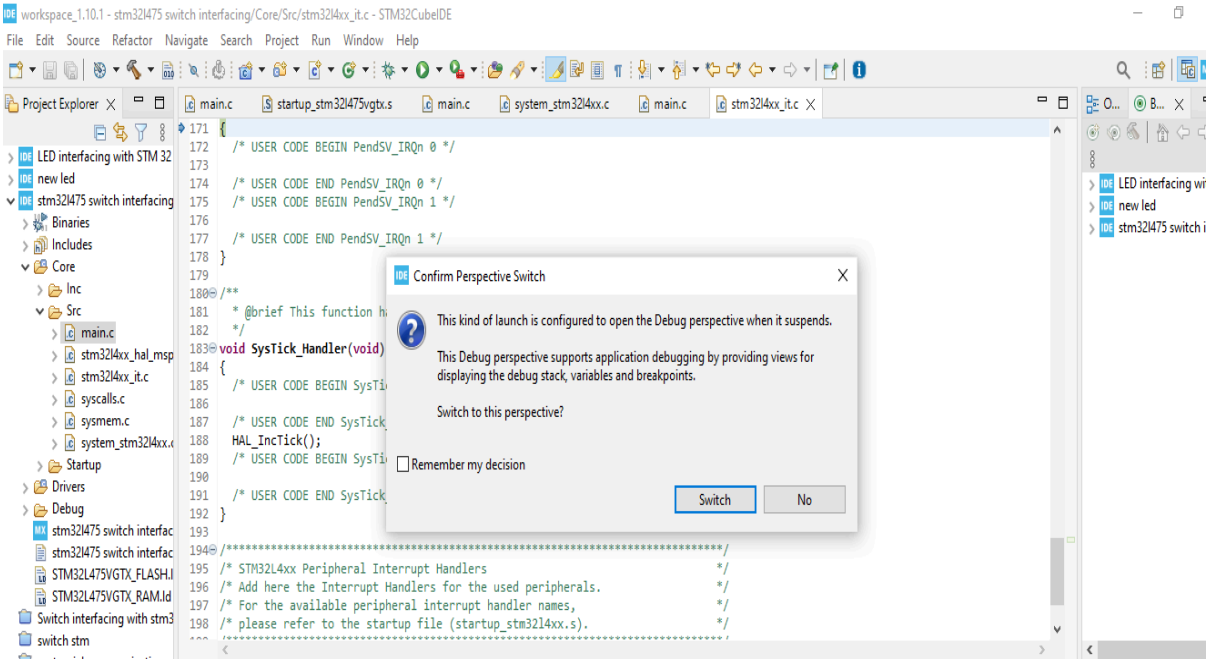

Fig: 1.8 – ‘switch’ on popups

14. Press resume, and the program will start executing.

OBSERVATION: Press the blue button to see the LED output. Pressing the blue button turns off the LED.

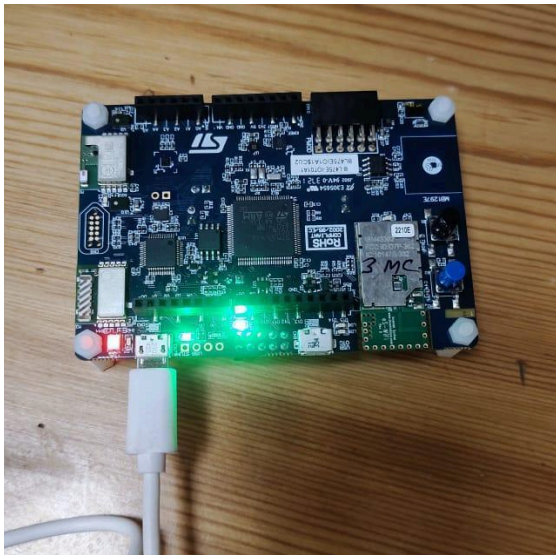

Fig: 1.8 – Output from experiment

Reflection Summary

1. When you press and release the blue user button (B1 on PC13), what do you see happening on the LED connected to PB14, and how long does it stay ON or OFF with your current program?
When the blue user button is pressed, PC13 becomes LOW and the LED on PB14 turns ON and when the button is released the LED turns OFF.
After the 100 ms delay, the LED is forced OFF, even if the button is still pressed.

2. If the LED sometimes flickers or does not behave as expected when you press the button quickly, what might be happening inside the button (mechanically and electrically), and how could you change your code or hardware to make the LED response more stable?
The LED flickring and not behaving as expected is due to mechanical contact bounce. It happens because the push button is not perfectly smooth inside. When the button is pressed, its metal contacts bounce rapidly for a few milliseconds instead of making it a one clean process. The microcontroller reads these tiny bounces as many quick responses, which causes the LED to behave unpredictably.
To fix this problem, a small delay can be added in the code to filter out the bounces. Another method that can be used to correct this problem is to add a small capacitor to the button so that the signal becomes smooth and stable.

3. Imagine you want your microcontroller to do many tasks (sensor reading, communication, display) while still reacting quickly to a button press. How could using interrupts on a GPIO pin help in such a situation compared to just checking the pin inside the main loop?
When a microcontroller is busy handling tasks like reading sensors, updating a display, or communicating with other devices, continuously checking the button in the main loop slows everything down. Polling the pin all the time also reduces how quickly the button press is detected.
Using a GPIO interrupt solves this problem. With an interrupt, the microcontroller freely focuses on other tasks, and the hardware will automatically trigger an immediate response the moment the button is pressed. This makes the system faster, more efficient, and much more responsive.